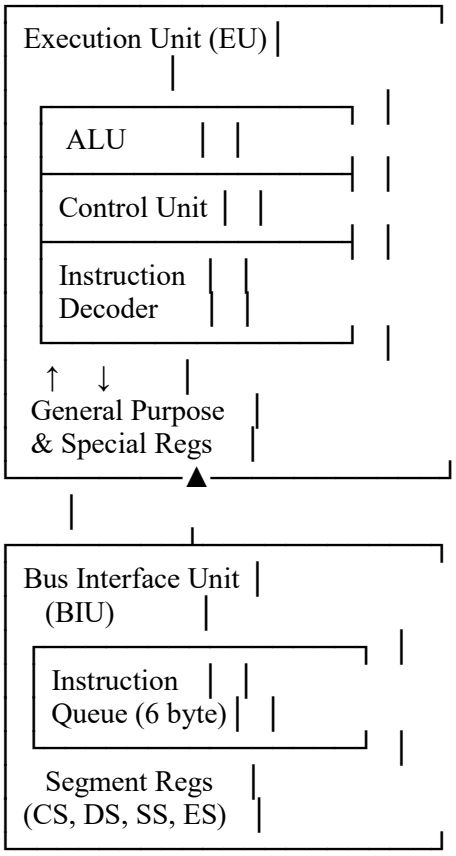
1: Explain the Architecture of 8086 Microprocessor with a Neat Block Diagram

Introduction:

The **Intel 8086 microprocessor** is a 16-bit microprocessor introduced in 1978. It has a 20-bit address bus and a 16-bit data bus, allowing it to access **1 MB of memory** and process **16 bits of data** at a time. The architecture is divided into two functional units:

- **Bus Interface Unit (BIU)**
- **Execution Unit (EU)**

Block Diagram:



Components:

1. Execution Unit (EU):

Responsible for fetching, decoding, and executing instructions.

- **Arithmetic and Logic Unit (ALU):** Performs arithmetic and logic operations.
- **Instruction Decoder:** Decodes fetched instructions.
- **Registers:**
  - **General Purpose Registers:** AX, BX, CX, DX (can be used as 8-bit or 16-bit)
  - **Pointer/Index Registers:** SP, BP, SI, DI
  - **Flag Register:** Indicates status of operations (e.g., Zero, Carry, Overflow).

- **Control Unit:** Directs internal operations.
- **Instruction Pointer (IP):** Holds offset address of the next instruction from the code segment.

Steps:

- EU fetches instruction from the queue.
- Decodes and prepares operands.
- Requests BIU to perform memory operations.
- Executes instruction.

2. Bus Interface Unit (BIU):

Handles all data, address, and instruction transfers on the system bus.

- **Fetches instructions in advance** and stores them in the 6-byte **instruction queue** (pipelining).
- Contains **Segment Registers**:
  - CS (Code Segment)
  - DS (Data Segment)
  - SS (Stack Segment)
  - ES (Extra Segment)
- Generates 20-bit **physical addresses** using:
- Physical Address = (Segment Address × 10H) + Offset
- Manages instruction prefetch and memory access.

Pipelining:

- **BIU** fetches instructions into queue while **EU** executes.
- Reduces idle time and increases speed (except in jumps/calls).

Key Features:

- 20-bit address bus → 1 MB memory space.
- 16-bit data bus → 8 or 16-bit data transfers.
- Segmented memory model for code, data, stack, and extra.
- Prefetch queue allows overlapping of fetch and execute cycles.

Summary:

The 8086 architecture separates instruction fetch and execution using BIU and EU, making it one of the early **pipelined architectures**. The use of segment registers and a strong instruction set allows efficient memory and I/O interfacing.

2: Describe the Various Addressing Modes of 8086 with Examples

Introduction:

Addressing modes define **how the operand is accessed** during instruction execution. The 8086 supports **rich and versatile addressing modes** to access data from registers, memory, or directly from the instruction.

1. Immediate Addressing Mode

Operand is given explicitly in the instruction.

Syntax: MOV AX, 0050H
→ Move immediate value 0050H to AX register.

2. Register Addressing Mode

Operand is a **register** (data is in a register).

Syntax: MOV AX, BX
→ Copy content of BX into AX.

3. Direct Addressing Mode

Operand is **located at a memory address**, which is directly specified.

Syntax: MOV AX, [1234H]
→ Move content of memory location 1234H into AX.

Effective Address = DS:1234H

4. Register Indirect Addressing Mode

Memory address of operand is **stored in a register**.

Allowed Registers: SI, DI, BX, BP (with segment DS or SS)

Syntax: MOV AX, [BX]
→ Move content at memory location pointed by BX into AX.

If DS = 5000H, BX = 1000H
→ Physical address = 50000H + 1000H = 51000H

5. Based Addressing Mode

Effective address is formed by adding a **base register (BX or BP)** to an offset.

Syntax: MOV AX, [BX + 04H]

6. Indexed Addressing Mode

Effective address = **Index Register (SI/DI) + offset**

Syntax: MOV AX, [SI + 08H]

Used commonly in array or string operations.

7. Based Indexed Addressing Mode

Effective address = **Base Register + Index Register**

Syntax: MOV AX, [BX + SI]

8. Based Indexed with Displacement

Effective address = **Base + Index + Displacement**

Syntax: MOV AX, [BX + SI + 05H]
→ Adds contents of BX, SI, and 5 to get the memory address.

Special Note:

- **Segment register (DS)** is used implicitly unless BP is used (then SS).
- **[] brackets** indicate memory location access.

Addressing Mode Comparison Table:

Mode	Example	Operand Location
Immediate	MOV AX, 00FFH	Inside instruction
Register	MOV AX, BX	CPU register
Direct	MOV AX, [1234H]	Memory address
Register Indirect	MOV AX, [BX]	Memory via register
Based	MOV AX, [BP+4]	Memory
Indexed	MOV AX, [SI+10]	Memory
Based Indexed	MOV AX, [BX+SI]	Memory
Based Indexed + Displacement	MOV AX, [BX+DI+10H]	Memory

Conclusion:

8086 provides **powerful and flexible addressing modes**, enabling efficient access to operands from memory or registers — making it ideal for system-level programming, OS design, and embedded systems.

3: Discuss the Features of 8086 Microprocessor

Introduction:

The **Intel 8086** is a 16-bit microprocessor developed in 1978. It was the **first member of the x86 architecture family**, supporting rich addressing and instruction modes, pipelining, and 1 MB memory access.

Key Features:

Feature	Description
Word Size	16-bit processor: processes 16-bit data at a time
Address Bus	20-bit address bus → can access $\mathbf{2^{20} = 1}$ **MB** memory
Data Bus	16-bit bidirectional data bus (can also handle 8-bit data)
Instruction Queue	6-byte prefetch queue for pipelining (overlaps fetch & execution)

Feature	Description
Registers	14 registers: general, segment, pointer/index, and flags
Segmented Memory	4 segments: Code (CS), Data (DS), Stack (SS), Extra (ES)
Modes of Operation	Minimum Mode (single CPU), Maximum Mode (multiprocessor)
Clock Frequency	Operates at 5, 8, or 10 MHz
I/O Ports	Can address up to **64K I/O ports**
Instruction Set	Rich instruction set including arithmetic, logical, control transfer
Flag Register	16-bit flag register: status flags (ZF, CF, SF...) & control flags (IF, DF, TF)
Interrupt Support	Supports **maskable and non-maskable interrupts (INTR, NMI)**
Memory Segmentation	64 KB per segment, enables modular programming

Advantages:

- Efficient pipelining boosts speed.
- Flexible addressing modes (register indirect, base+index, etc.)
- Modular and multitasking friendly due to segmented memory.
- Widely used in embedded systems, early PCs, and industrial controls.

4: Differentiate Between Minimum Mode and Maximum Mode Operation of 8086

Introduction:

The **8086 operates in two modes**, selected using the **MN/M$\overline{X}$ pin**:

- **Minimum Mode:** Single processor system
- **Maximum Mode:** Multiprocessor or coprocessor system

Differences:

Aspect	Minimum Mode	Maximum Mode
Use-case	Single microprocessor system	Multiprocessor or coprocessor system
Control Signal Generator	8086 itself	Requires external Bus Controller (8288)
Pin MN/M$\overline{X}$	Logic 1 (HIGH)	Logic 0 (LOW)
Control Signals Provided By	8086	8288 Bus Controller
Signals	ALE, W$\overline{R}$, R$\overline{D}$, INT$\overline{A}$, DEN, DT/$\overline{R}$	S2, S1, S0 (status), INT$\overline{A}$, MRD$\overline{C}$, IOW$\overline{C}$
Queue Status Signals	Not available	QS0, QS1 are available
Hold Acknowledge Signals	HOLD, HLDA	RQ/GT0, RQ/GT1
Timing Diagrams	Simple	Complex, due to external controller
Complexity	Low	Higher, due to extra ICs

Block Diagrams (Simplified):

Minimum Mode:

[8086]

↓
[Memory & I/O]

Maximum Mode:

[8086] → [8288 Bus Controller]
↓
[Memory & I/O Devices]

Conclusion:

- Use **Minimum Mode** for simpler applications with a single processor.
- Use **Maximum Mode** for multiprocessor environments where 8086 needs to share system resources with other processors or coprocessors (e.g., 8087).

5: Explain Memory Segmentation in 8086

Introduction:

8086 uses a **segmented memory model** to efficiently address a **1 MB memory space** using 16-bit registers.

Why Segmentation?

- 8086 has only **16-bit registers** (max value 65535), so it can't directly access 1 MB (2^{20}).
- To overcome this, memory is divided into **segments**.

Types of Segments:

Segment	Register	Function
Code Segment	CS	Stores program instructions
Data Segment	DS	Holds global/static data
Stack Segment	SS	Used for function calls, local variables
Extra Segment	ES	Used for string operations or extra data

Segment Size:

- Each segment = 64 KB max
- 8086 uses a **20-bit physical address** generated as:

Physical Address = Segment Register × 10H + Offset (from IP, SI, DI, etc.)

Example:

CS = 2000H, IP = 1000H
Physical Address = 20000H + 1000H = 21000H

Segment + Offset Mechanism:

- Segment register defines base (start of segment)
- Offset register defines displacement within the segment

Segment	Paired Offset
CS	IP
DS	SI, DI
SS	SP, BP
ES	DI

Advantages:

- Logical separation of code, data, and stack
- Enables modular and relocatable programs
- Increases flexibility and security

Diagram (Conceptual):

```
Memory (1 MB)
├── Code Segment (64 KB)
├── Data Segment (64 KB)
├── Stack Segment (64 KB)
└── Extra Segment (64 KB)
```

Summary:

Memory segmentation in 8086 enhances **addressing flexibility**, enables **efficient memory utilization**, and supports **program modularity** despite hardware limitations of 16-bit address registers.

6: Describe the Physical Memory Organization in 8086

Introduction:

8086 microprocessor can **physically address up to 1 MB** of memory using a **20-bit address bus**, even though it has **16-bit registers**. The physical memory is structured in a **segmented architecture**.

Memory Size:

- **Address Bus** = 20 bits → 2^{20} = 1,048,576 bytes = 1 MB memory.
- **Data Bus** = 16 bits → Can transfer 16-bit or 8-bit data at a time.

Memory Banks: 8086 organizes memory as **two banks**:

1. **Even Address Bank** – for even addresses
2. **Odd Address Bank** – for odd addresses

Each memory location is 1 byte wide, but since 8086 can read/write **2 bytes in one cycle**, it uses:

- **BH$\overline{E}$ (Bus High Enable)** and
- **A0** to control **which bank to access**.

Memory Bank Access Mechanism:

Address	A0	BHĒ	Access
Even	0	1	Lower byte (D0–D7)
Odd	1	0	Upper byte (D8–D15)
Word (Even Address)	0	0	Both banks
Word (Odd Address)	1	0	2 bus cycles required

Address Lines:

- **A0–A19** form the **20-bit address**
- **A0** differentiates low/high byte
- **A1–A19** used to locate the actual memory location

Memory Access Optimization:

- For 16-bit transfers, 8086 attempts to align data to **even addresses**.
- Misaligned 16-bit data (e.g., at 0003H) takes **two memory access cycles**.

Example: To access 16-bit data at address 0004H:

- A0 = 0 → selects even bank
- BH$\overline{E}$ = 0 → selects odd bank
- Both bytes can be read simultaneously.

Memory Segments:

- **64 KB segments**: Code, Data, Stack, Extra
- Overlap is allowed: Multiple segments can point to the same physical memory

Diagram:

```
+-------------------------+ ← 0FFFFFH (1 MB)
|     Extra Segment       |
+-------------------------+
|     Stack Segment       |
+-------------------------+
|     Data Segment        |
+-------------------------+
|     Code Segment        |
+-------------------------+ ← 00000H
```

Conclusion:

Physical memory in 8086 is organized using **banked access and segmented architecture**, maximizing both **efficiency** and **addressing flexibility** within 1 MB memory space.

7: Write Short Notes on I/O Addressing Capabilities

Introduction:

The 8086 microprocessor can communicate with external devices (keyboard, display, etc.) using **Input/Output (I/O) ports**. These ports are addressed separately from memory using **I/O addressing**.

I/O Addressing Types:

1. **Isolated I/O (Port Mapped I/O)**
2. **Memory-Mapped I/O**

1. Isolated I/O (Port Mapped I/O):

- **Separate address space** for I/O (not part of main memory).
- **IN** and **OUT** instructions used.
- **16-bit I/O address bus** → can access **64K I/O ports (0–FFFFH)**.
- Faster and more efficient for small systems.

Example:

```
IN AL, 60H   ; Read from keyboard port
OUT 64H, AL  ; Send data to display controller
```

2. Memory-Mapped I/O:

- I/O devices are **treated as memory locations**.
- Use **MOV** instructions for access.
- Slower, but **more flexible**.
- **Same address bus** and control signals as memory.

Example: MOV AL, [1000H] ; Read from memory-mapped device

I/O Addressing Signals:

Signal	Description
IO/$\overline{M}$	Distinguishes I/O and memory (1 = I/O)
R$\overline{D}$ / W$\overline{R}$	Read/Write control
BH$\overline{E}$	High byte enable
A0	Selects low/high byte port

Summary:

8086 can efficiently communicate with **64K I/O ports** using isolated or memory-mapped I/O. It provides **flexible and high-speed access** to external devices, making it suitable for embedded applications.

8: Explain the Bus Interface Unit (BIU) and Execution Unit (EU) of 8086

Introduction:

The 8086 processor is divided into two functional blocks:

- **Bus Interface Unit (BIU)**
- **Execution Unit (EU)**

This design supports **pipelining**, where fetching and executing instructions can overlap.

1. Bus Interface Unit (BIU):

Functions:

- **Fetches instructions** from memory
- Stores in the **6-byte instruction queue**
- **Calculates physical addresses** (Segment × 10H + Offset)
- Handles **address, data, and control buses**

Components:

- **Instruction Queue (6 bytes)** → enables pipelining
- **Segment Registers**:
  - **CS**: Code Segment
  - **DS**: Data Segment
  - **SS**: Stack Segment
  - **ES**: Extra Segment
- **Instruction Pointer (IP)** → Offset address within Code Segment

Example Address Calculation:

CS = 2000H, IP = 0100H → Physical Address = 20000H + 0100H = 20100H

2. Execution Unit (EU):

Functions:

- Decodes and executes instructions
- Performs **arithmetic and logical operations**
- Requests BIU for memory accesses

Components:

- **ALU (Arithmetic Logic Unit)** → Performs operations like ADD, SUB, AND, OR
- **Control Unit** → Coordinates instruction execution
- **Registers**:
  - **General Purpose:** AX, BX, CX, DX
  - **Pointer & Index:** SP, BP, SI, DI
  - **Flag Register:** CF, ZF, SF, OF, etc.

Working Mechanism:

Step	EU	BIU
1	Decodes current instruction	Fetches next instruction
2	Executes using registers	Queues instructions
3	Operands accessed/modified	Addresses calculated

Diagram (Conceptual):

```
+-------------------+
| Execution Unit   |
| - ALU            |
| - Registers      |
| - Control Unit   |
+--------▲----------+
         |
+--------┴----------+
| Bus Interface Unit|
| - IP, Queue      |
| - Segment Regs   |
+-------------------+
```

Summary:

- The **BIU** focuses on instruction prefetching, memory addressing, and bus control.
- The **EU** decodes and executes the instructions.
- This split architecture improves performance through **instruction pipelining**, a hallmark of 8086's efficiency.

9: Write an Assembly Program to Find the Square of a Number (8086)

Objective:

To write an 8086 Assembly Language Program (ALP) that computes the square of an 8-bit number and stores the result in memory.

Assumptions:

- Input number is **stored at memory location** 3000H
- Result (square) is stored at memory location 3001H (lower byte) and 3002H (higher byte) since the square of an 8-bit number can be up to 16 bits.

Program (Using 8086 Instructions):

```
.model small
.data
   num db 07H        ; Example number (7 decimal)
   result dw ?       ; Reserve 2 bytes for result

.code
start:
   mov ax, @data
   mov ds, ax        ; Initialize data segment

   mov al, num       ; Load the number into AL
   mov ah, 0         ; Clear AH
   mul al            ; Multiply AL × AL, result in AX

   mov result, ax    ; Store result in memory

   ; End program
   mov ah, 4ch
   int 21h
```

end start

Explanation:

- AL × AL → AX (result stored in AX register)
- MUL AL performs **unsigned multiplication**
- Result: AX = AL × AL (16-bit result)

Result Interpretation:

If input = 07H (7),
then 07H × 07H = 49 (031H)
AX = 0031H, stored in result

10: Explain the General Bus Operation of 8086

Introduction:

The **bus system** of 8086 is responsible for transferring data, address, and control signals between the processor and memory/I/O devices. The **general bus operation** refers to how 8086 manages **memory and I/O access** via time-multiplexed buses.

Buses Used:

1. **Address Bus (20-bit)** → A0–A19
2. **Data Bus (16-bit)** → D0–D15
3. **Control Bus** → R$\overline{D}$, W$\overline{R}$, ALE, etc.

Multiplexed Bus:

- **A0–A15** are multiplexed with **D0–D15**
- **A16–A19** are multiplexed with status signals (S3–S6)

Bus Cycle Components:

1. **T1 State:**
   - Address placed on the bus
   - **ALE** (Address Latch Enable) is HIGH → Address is latched
2. **T2 State:**
   - Bus is tristated for data transfer
   - Control signals activated (e.g., **R$\overline{D}$** or **W$\overline{R}$**)
3. **T3 State:**
   - Data read from or written to memory/I/O
4. **T4 State:**
   - Bus signals return to idle state

Read/Write Signals:

Signal	Purpose
R$\overline{D}$	Read Enable (Low active)
W$\overline{R}$	Write Enable (Low active)
ALE	Indicates valid address on the bus

Signal	Purpose
DT/$\overline{R}$	Direction of data flow
DEN	Data Enable (activates transceivers)

I/O and Memory Operations:

Operation	Signals Used
Memory Read	R$\overline{D}$, ALE, M/I$\overline{O}$
Memory Write	W$\overline{R}$, ALE, M/I$\overline{O}$
I/O Read	R$\overline{D}$, ALE, IO/$\overline{M}$
I/O Write	W$\overline{R}$, ALE, IO/$\overline{M}$

Diagram (Simplified Timing):

T1: Address (A0–A15) on bus → ALE ↑
T2: Address removed → Control signals set (R$\overline{D}$/W$\overline{R}$)
T3: Data read/written
T4: Signals reset

Summary:

The **general bus operation** in 8086 is a 4-T state machine that handles **data/address multiplexing**, manages control signals, and supports synchronous communication with memory and I/O devices.

11: Discuss the Machine Language Instruction Formats in 8086

Introduction:

Each instruction in 8086 is stored in **machine language format** (binary). The **instruction format** varies depending on the type of instruction, operands used, and addressing mode.

Types of Instruction Formats:

Format Type	Description
One-byte format	Opcode only (e.g., CLC, NOP)
Two-byte format	Opcode + operand (e.g., immediate data)
Three-byte format	Opcode + address or displacement
Variable length	May include MOD-REG-R/M byte, displacement, or data

Key Components:

1. Opcode Byte:

- Specifies the operation to be performed
- Example: 100010 = MOV

2. MOD-REG-R/M Byte:

- Used when at least one operand is in memory
- **MOD** (2 bits): addressing mode
- **REG** (3 bits): destination/source register
- **R/M** (3 bits): register/memory operand

MOD	Description
00	Memory (no displacement)
01	Memory (8-bit disp)
10	Memory (16-bit disp)
11	Register mode

3. Displacement/Immediate Data:

- Displacement = address offset
- Immediate data = actual data in instruction

Example Instruction Format:

MOV AX, [BX+SI]

Opcode → 100010
MOD-REG-R/M → MOD=00, REG=000 (AX), R/M=000 (BX+SI)

Encoding Breakdown:

- Opcode: Specifies MOV
- MOD: 00 = Memory mode, no displacement
- REG: 000 = AX
- R/M: 000 = [BX + SI]

Typical Instruction Format:

Opcode	MOD-REG-R/M byte	Displacement	Immediate Data
1 Byte	1 Byte	1 or 2 Bytes	1 or 2 Bytes

Summary:

Instruction formats in 8086 are **variable-length**, optimized for flexible and compact encoding. The use of **MOD-REG-R/M bytes, immediate fields, and displacement** allows 8086 to support a wide range of instructions and addressing modes efficiently.

12: Compare the Various Instruction Types Available in 8086

Introduction:

The 8086 microprocessor supports a **rich instruction set** categorized by the type of operation performed. These instructions are classified into groups for easier programming and hardware efficiency.

Categories of Instruction Types:

Instruction Type	Description / Purpose	Example(s)
Data Transfer	Move data between registers, memory, I/O	MOV, PUSH, POP, XCHG
Arithmetic	Perform arithmetic operations	ADD, SUB, MUL, INC, DEC
Logical	Perform logical/bitwise operations	AND, OR, XOR, NOT, TEST
Control Transfer	Change sequence of execution	JMP, CALL, RET, LOOP, JC, JZ
String Operations	Process sequences of bytes/words	MOVSB, LODSB, STOSB, CMPSB
Processor Control	Alter flag or system state	CLC, STC, CLI, STI, HLT, WAIT
Flag Manipulation	Set or clear processor flags	CMC, SAHF, LAHF
Shift and Rotate	Bit manipulation	SHL, SHR, ROL, ROR
Interrupt Instructions	Trigger or return from interrupts	INT, IRET, INTO

Example Comparisons:

Category	Instruction	Effect
Data Transfer	MOV AX, BX	Copy BX to AX
Arithmetic	ADD AX, 05	AX ← AX + 05
Logical	AND AL, 0F	AL ← AL AND 0F
Control Transfer	JZ label	Jump to label if ZF=1
String	MOVSB	Move byte from DS:SI to ES:DI
Processor Control	CLI	Clear Interrupt Flag (IF=0)

Summary:

8086 offers a **powerful and flexible instruction set** that covers every essential operation — from basic data handling to complex control structures, making it ideal for both systems-level and application-level programming.

13: Explain Register Addressing Mode and Give Examples

Introduction:

Register addressing mode is one of the simplest and fastest modes in 8086 where **both the operands are registers**.

Syntax:

MOV AX, BX

This copies the contents of register **BX** into register **AX**.

Characteristics:

- Operands are internal CPU registers.
- No memory access required → very fast.
- Instruction size is small (1 or 2 bytes).
- Operands can be 8-bit (AH, AL, BH...) or 16-bit (AX, BX, CX...).

8-bit Register Example:

MOV AL, BL ; Move contents of BL into AL

16-bit Register Example:

ADD DX, CX ; DX ← DX + CX

Instructions Using Register Addressing:

- MOV, ADD, SUB, CMP, XCHG, AND, OR, XOR, etc.

Advantages:

- Faster execution
- Low memory usage
- Simplifies operand fetching

Summary:

Register addressing mode is highly efficient for internal operations, making it ideal for loops, counters, arithmetic, and logic operations.

14: Explain Signal Description of 8086 with Pin Diagram

Introduction:

The 8086 microprocessor is available in a **40-pin DIP package**. It has **different sets of control, data, and address lines**. Signals vary based on mode (minimum or maximum).

Pin Diagram of 8086:

```
  ┌──────────┐
AD0 │ 1   40 │ VCC
AD1 │ 2   39 │ GND
AD2 │ 3   38 │ NMI
AD3 │ 4   37 │ INTR
AD4 │ 5   36 │ CLK
AD5 │ 6   35 │ READY
AD6 │ 7   34 │ RESET
AD7 │ 8   33 │ TEST
AD8 │ 9   32 │ MN/M$\overline{X}$
AD9 │ 10  31 │ HOLD
... │      ... │
  └──────────┘
```

Signal Categories:

1. Address/Data Bus (AD0–AD15):

- **Time-multiplexed**: Carries address during T1, data during T2–T4.
- Latches are used externally to separate address and data.

2. Address Bus (A16–A19) / Status Lines (S3–S6):

- Multiplexed with high address bits
- S3–S6 provide status information in maximum mode

3. Control Signals:

- **R$\overline{D}$**: Read control (active low)
- **W$\overline{R}$**: Write control (active low)
- **ALE**: Address Latch Enable (for demultiplexing address/data lines)
- **DEN**: Data Enable (enables external data transceivers)
- **DT/$\overline{R}$**: Direction control for data bus

Other Important Signals:

Signal	Description
RESET	Initializes CPU and clears flags
READY	Waits for slower memory or I/O devices
CLK	System clock input
INTR / NMI	Maskable and non-maskable interrupts
HOLD / HLDA	Bus request and acknowledgement
TEST	Used for WAIT instruction (for coprocessor testing)

Minimum vs Maximum Mode:

Mode	Signal Behavior
Minimum Mode	ALE, R$\overline{D}$, W$\overline{R}$, INT$\overline{A}$
Maximum Mode	Uses 8288 bus controller, generates S2, S1, S0

Summary:

The 8086 has **multiplexed pins** to handle address, data, and control efficiently. Pin functions adapt based on **mode of operation**, ensuring flexibility for single or multiprocessor systems.

1: Write an Assembly Language Program to Add Two 16-bit Numbers

Objective:

To write an ALP that adds two **16-bit numbers** stored in memory and stores the result in another memory location.

Assumptions:

- First number stored at 3000H
- Second number stored at 3002H
- Result (16-bit) stored at 3004H

Program:

```
.model small
```

```
.data
   num1 dw 1234H       ; First 16-bit number
   num2 dw 0F12H       ; Second 16-bit number
   result dw ?         ; 16-bit result

.code
start:
   mov ax, @data
   mov ds, ax          ; Initialize data segment

   mov ax, num1        ; Load num1 into AX
   add ax, num2        ; Add num2 to AX
   mov result, ax      ; Store result in memory

   ; Exit
   mov ah, 4ch
   int 21h
end start
```

Explanation:

- AX is used to hold the result of the addition.
- The ADD instruction updates flags like CF (Carry Flag) in case of overflow.
- Operands and results are stored in memory using DW (Define Word).

2: Write an ALP to Perform Multiplication and Store Result in Memory

Objective:

Multiply two 8-bit or 16-bit numbers and store the 16-bit or 32-bit result respectively.

Program for 8-bit Multiplication:

```
.model small
.data
   num1 db 06h
   num2 db 07h
   result dw ?

.code
start:
   mov ax, @data
   mov ds, ax

   mov al, num1        ; Load first 8-bit number into AL
   mov bl, num2        ; Load second number into BL
   mul bl              ; AL × BL → AX
   mov result, ax      ; Store 16-bit result

   mov ah, 4ch
   int 21h
end start
```

Program for 16-bit Multiplication:

```
.model small
.data
   num1 dw 1234H
   num2 dw 0002H
   result1 dw ?
   result2 dw ?        ; For upper 16 bits

.code
start:
   mov ax, @data
   mov ds, ax

   mov ax, num1
   mov bx, num2
   mul bx              ; AX × BX → DX:AX
   mov result1, ax
   mov result2, dx

   mov ah, 4ch
   int 21h
end start
```

Explanation:

- MUL is unsigned multiplication.
- For 8-bit: AL × operand → AX
- For 16-bit: AX × operand → DX:AX
- Results stored in memory using DW.

3: Explain Stack Structure in 8086 and Its Operations

Introduction:

The **stack** in 8086 is a **LIFO (Last-In-First-Out)** data structure used for **function calls, parameter passing, and temporary storage**. It operates using **SS (Stack Segment)** and **SP (Stack Pointer)** registers.

Key Components:

Register	Purpose
SS	Holds base address of stack segment
SP	Holds offset address (grows downward)
BP	Used to access stack contents

Physical Address:

Physical Stack Address = SS × 10H + SP

Stack Operations:

1. PUSH:

- Decrements SP by 2
- Stores word at location [SS:SP]

Example: PUSH AX ; Stack[SP-2] ← AX, SP ← SP - 2

2. POP:

- Retrieves word from [SS:SP]
- Increments SP by 2

Example: POP AX ; AX ← Stack[SP], SP ← SP + 2

Stack Memory Growth:

- Stack grows **downward** from high to low addresses.
- Stack is usually initialized with SP pointing to the **top of the stack segment**.

Diagram (Conceptual):

```
High Address (e.g., 0FFFH)
↓
| -----> PUSH → SP = SP - 2
| Data
| Data
↑
| -----> POP  → SP = SP + 2
Low Address
```

Function Call Example:

When a CALL instruction is used:

- The return address is **pushed** onto the stack
- After the procedure finishes, RET pops the return address from stack

Summary:

The **stack structure** in 8086 is crucial for **nested subroutine calls**, **interrupts**, and **temporary data storage**. Stack operations rely on **SS:SP** pair and are central to structured programming in assembly.

4: Describe Interrupt Cycle and Types of Interrupts in 8086

What is an Interrupt?

An **interrupt** is a mechanism that **temporarily halts** the execution of the current program to **attend to a higher priority task**. Once the task is serviced, control returns to the original program.

Interrupt Cycle of 8086:

When an interrupt is triggered, the 8086 executes the **interrupt acknowledge cycle**:

Steps in the Interrupt Cycle:

1. **Save Context**:
   - Pushes FLAGS, CS, and IP onto the stack.
2. **Disable Further Interrupts**:
   - Clears the Interrupt Flag (IF = 0).
3. **Fetch Interrupt Vector**:
   - Vector address = Interrupt Number × 4
   - Obtains IP and CS from Interrupt Vector Table (IVT).
4. **Transfer Control**:
   - Loads IP and CS → jumps to the Interrupt Service Routine (ISR).
5. **Resume Execution**:
   - IRET instruction restores flags, CS, and IP to resume execution.

Types of Interrupts in 8086:

Type	Triggered By	Vector Address	Example
Hardware	External devices	INTR/NMI pins	Keyboard
Software	INT n instruction	Defined in code	INT 21H
Internal	Exceptions, errors	Automatically triggered	Divide by zero

Hardware Interrupts:

- **INTR (Interrupt Request)**: Maskable (controlled by IF)
- **NMI (Non-Maskable Interrupt)**: Cannot be disabled

Software Interrupts:

- Invoked using INT n, e.g., INT 10H, INT 21H
- Widely used for BIOS and DOS function calls.

Internal Interrupts:

- Triggered by special conditions:
  - Divide-by-zero → Type 0
  - Overflow → Type 4

Summary:

The interrupt cycle allows 8086 to **respond to asynchronous events** efficiently. Interrupt types include **hardware**, **software**, and **internal**, each with specific handling mechanisms and vector locations.

5: Explain Interrupt Service Routine with Types and Vectors

Interrupt Service Routine (ISR):

An **ISR** is a special subroutine that is executed **in response to an interrupt**. Once triggered, the 8086 jumps to the address of the ISR using the interrupt vector.

Steps in Executing ISR:

- **Push flags, CS, IP** to stack
- **Load CS:IP from vector table**
- **Execute ISR**
- **Use IRET** to return (restores CS, IP, Flags)

Interrupt Vector Table (IVT):

- Located at memory **0000H to 03FFH**
- Contains **256 vectors**, each 4 bytes (2 bytes IP, 2 bytes CS)
- Address of ISR = Interrupt Number × 4

Example: INT 21H (DOS Interrupt)

- Used for input/output operations
- If AH = 09H → Display string

Types of ISRs:

Type	Description	Triggered By	Vector Address
Type 0	Divide Error	Division by zero	0000H
Type 1	Single Step	Trap flag enabled	0004H
Type 2	NMI	Non-maskable	0008H
Type 3	Breakpoint	INT 3 instruction	000CH
Type 4	Overflow	ON INTO instruction	0010H

Software ISR Examples:

INT Instruction	Purpose
INT 10H	Video services
INT 13H	Disk I/O services
INT 16H	Keyboard input
INT 21H	DOS functions

Summary:

ISRs are essential for **interrupt handling** in real-time and system-level programming. The 8086 uses a fixed **interrupt vector table** to locate ISRs, and IRET to return from them safely.

6: Compare Maskable and Non-Maskable Interrupts with Examples

Introduction:

8086 supports two primary hardware interrupts:

- **INTR**: Maskable
- **NMI**: Non-Maskable

Comparison Table:

Feature	Maskable Interrupt (INTR)	Non-Maskable Interrupt (NMI)
Trigger Mechanism	External hardware signal	Dedicated NMI pin
Interrupt Flag (IF)	Must be 1 (enabled)	Not affected by IF
Can be Disabled?	Yes (using CLI)	No
Priority	Lower	Highest
Vector Location	Supplied by device	Fixed at 0008H
Acknowledgement	Requires INTĀ	No acknowledgement needed
Use Cases	Keyboard, mouse, printer	Power failure, memory parity error
Examples	Keyboard interrupt	Emergency shutdown signal

Diagram:

```
| External Device  | ——INTR—> |                |
| (e.g., Keyboard) |        |   8086 CPU      |
        |               |
                        |  —> Vectored ISR |
+------------------+ ——NMI—>  |                |
| Emergency Alert  |
```

Summary:

- **Maskable interrupts (INTR)** are general-purpose and can be turned off.
- **Non-maskable interrupts (NMI)** are reserved for **critical failures** and **cannot be disabled**.
- Both play a key role in **real-time system responsiveness**.

7: Explain the Use of Assembler Directives and Operators

What are Assembler Directives?

Assembler directives are instructions to the assembler, not the CPU. They help the assembler **organize code, allocate memory, define constants**, and control program structure — but they are **not converted into machine code**.

Common Assembler Directives in 8086:

Directive	Purpose	Example
.MODEL	Defines memory model	.MODEL SMALL
.DATA	Starts data segment	.DATA
.CODE	Starts code segment	.CODE
.STACK	Defines stack segment	.STACK 100H
DB	Define Byte	val DB 10H
DW	Define Word (16-bit)	num DW 1234H
EQU	Defines constants	PI EQU 3.14
ORG	Sets the origin (offset)	ORG 1000H
END	Marks end of program	END start
ASSUME	Assigns segment registers	ASSUME CS:CODE, DS:DATA

Operators in Assembly:

Operator	Function	Example
+ / -	Arithmetic	MOV AX, BX + 10
*	Multiplication	MOV AX, 2 * 4
:	Label / segment override	start:
$	Current address	MOV CX, OFFSET $
OFFSET	Gets offset of variable	MOV SI, OFFSET arr
PTR	Specifies type of data	MOV BYTE PTR [BX], 0Fh
SEG	Gets segment address	MOV AX, SEG var

Importance:

- Makes code **readable and maintainable**
- Provides **structure and control** during program assembly
- Helps with **modular and memory-efficient** programming

8: Explain the Interrupt Acknowledgment Bus Cycle

What is an Interrupt Acknowledgement Cycle?

When a **hardware interrupt** (INTR) is received, the 8086 acknowledges it through a **special bus cycle** to read the **interrupt vector number** from the device.

Sequence for INTR (Maskable Interrupt):

- **INTR Line High** → External device requests interrupt.
- **IF = 1** (Interrupt Flag is enabled) → CPU accepts interrupt.
- After completing current instruction, 8086 initiates **2 interrupt acknowledge (INTĀ) bus cycles**.

INT Acknowledge Bus Cycle:

Cycle	Action
INTĀ 1	Tells the device the interrupt is recognized.
INTĀ 2	External device places the **interrupt type number** on data bus.

Interrupt Type = 8-bit value (e.g., 08H, 09H)

Internal Flow:

- CPU saves current FLAGS, CS, IP.
- Loads new CS:IP from **Interrupt Vector Table** at:
- Address = Type × 4
- Jumps to ISR.

Example:

If device sends **INT Type 09H**,
→ Vector address = 09H × 4 = 24H
→ CPU loads IP and CS from addresses 0024H and 0026H.

Summary:

The **interrupt acknowledgment cycle** is vital for communicating with external devices. It allows the CPU to **identify the interrupt source** and **redirect execution** using vectored addressing.

9: Describe the Sequence of Operations During an Interrupt in 8086

Overview:

When an interrupt occurs, 8086 temporarily suspends current execution, **services the interrupt**, and resumes the previous program. This process is well-structured and involves saving and restoring CPU state.

Sequence of Events:

- **Interrupt Triggered**:
    1. Can be software (INT n), hardware (INTR/NMI), or internal (exception)
- **Check IF (for INTR)**:
    1. If IF = 1, maskable interrupts are allowed.
- **Complete Current Instruction**:
    1. CPU finishes the current instruction before handling the interrupt.
- **Push Flags, CS, IP onto Stack**:
    1. Saves the context to resume later.
- **Clear Trap and Interrupt Flags**:
    1. Disables further interrupts (IF = 0, TF = 0)
- **Fetch ISR Address from Vector Table**:
    1. ISR Address = Type × 4
    2. Loads new IP and CS
- **Execute Interrupt Service Routine (ISR)**
- **Return Using IRET**:
    1. Restores IP, CS, and FLAGS from stack

Diagram: Interrupt Handling Flow

[INTR] → [Check IF] → [Push FLAGS, CS, IP] →
[Disable IF, TF] → [Fetch ISR address] →
[Execute ISR] → [IRET → Restore & Resume]

Example:

INT 09H ; Keyboard ISR

- Pushes context
- Loads CS:IP from 0024H & 0026H
- Executes ISR
- IRET returns to the main program

Summary:

The 8086 interrupt handling sequence ensures **safe and organized task switching**, allowing asynchronous events (e.g., I/O, system errors) to be handled efficiently without corrupting the execution state.

10. Short Notes on Different Types of Interrupts (Type 0 to Type 4)

The 8086 microprocessor has a well-defined **interrupt mechanism**, with interrupt types ranging from **Type 0 to Type 255**, each associated with a **4-byte interrupt vector** in the **Interrupt Vector Table (IVT)** located from memory address 00000H to 003FFH.

Type 0 – Divide-by-Zero Interrupt

- **Trigger**: Occurs when a divide instruction tries to divide a number by zero.
- **Cause**: Division error due to zero divisor or a result too large to store.
- **Interrupt Vector Address**: Type 0 × 4 = 0000H
- **Use**: To handle programming errors in division operations.
- **Priority**: Highest priority interrupt.

Type 1 – Single-Step Interrupt (Trap Interrupt)

- **Trigger**: When the **Trap Flag (TF)** in the FLAGS register is set.
- **Purpose**: Enables execution of one instruction at a time for **debugging**.
- **ISR Address**: 00004H
- **Use**: Displays all registers; allows the programmer to trace code.

Type 2 – Non-Maskable Interrupt (NMI)

- **Trigger**: External signal at the **NMI pin**.
- **ISR Address**: 00008H
- **Purpose**: Handles **critical failures** like power loss, memory parity error.
- **Cannot be disabled** by software; highest hardware interrupt priority.

Type 3 – Breakpoint Interrupt

- **Trigger**: Software instruction INT 3 or INT 03H.
- **ISR Address**: 0000CH
- **Use**: For **setting breakpoints** in large programs.
- **Debugging Aid**: Pauses execution and shows system status.

Type 4 – Overflow Interrupt

- **Trigger**: INTO instruction and **Overflow Flag (OF)** is set.
- **ISR Address**: 00010H
- **Purpose**: Detects **overflow errors in signed arithmetic**.
- **Use**: Prevents erroneous results from signed operations.

11. Illustrate Programming Using Assembler with an Example

Assembly language programming for 8086 involves writing low-level instructions using an **assembler** such as MASM or TASM. Programs include **instructions**, **assembler directives**, and **labels/comments**.

Structure of Assembly Program

```
.MODEL SMALL
.STACK 100H
.DATA
```

```
   NUM1 DB 25H
   NUM2 DB 30H
   RESULT DB ?
.CODE
START:
   MOV AX, @DATA       ; Load segment address
   MOV DS, AX

   MOV AL, NUM1        ; Load first number
   ADD AL, NUM2        ; Add second number
   MOV RESULT, AL      ; Store result

   MOV AH, 4CH         ; Terminate program
   INT 21H
END START
```

Explanation

- .MODEL SMALL: Indicates small memory model (64KB for code and data).
- .DATA, .CODE, .STACK: Define memory segments.
- MOV and ADD: Basic instructions for data transfer and arithmetic.
- INT 21H: DOS interrupt for terminating the program.

Assembler Directives Used

- .MODEL, .DATA, .CODE, .STACK, END
- These help the assembler understand structure, not converted into machine code.

12. Explain Single Step and Breakpoint Interrupts in 8086 (Extremely Detailed)

Single Step Interrupt (Type 1 Interrupt)

Also known as **Trap Interrupt**, this is an **internal** interrupt used for **debugging**.

How it Works:

- **Trap Flag (TF)** is set in the **FLAGS register**.
- After execution of each instruction, an interrupt of **Type 1** is triggered.
- The processor pushes **FLAGS**, **CS**, and **IP** onto the stack.
- It **jumps to the ISR** located at memory address 00004H.

Use Case:

- Executes one instruction and pauses.
- Useful for **debuggers** or tools like **soft ICE**.
- Helps analyse CPU **registers** and **memory** after each step.

Key Point from Document:

"The microprocessor pauses after executing every instruction. This is very useful during debugging".

Breakpoint Interrupt (Type 3 Interrupt)

This is a **software interrupt** triggered by inserting the instruction INT 03H into the code.

How it Works:

- INT 03H executes and invokes the ISR at 0000CH.
- The CPU pushes FLAGS, CS, and IP onto the stack.
- Control is transferred to the breakpoint ISR.
- The ISR can:
    1. Show values of registers.
    2. Modify memory.
    3. Resume execution after analysis.

Use Case:

- Used for placing **breakpoints** at specific addresses in a large program.
- Pauses execution to examine system status.

Key Point from Document:

"Used to cause breakpoints in the program... very useful in debugging large programs where single stepping is inefficient".

Comparison of Single Step and Breakpoint Interrupts

Feature	Single Step (Type 1)	Breakpoint (Type 3)
Trigger	Setting Trap Flag (TF)	INT 03H instruction
Purpose	Step-by-step execution	Breakpoint at specific location
ISR Address	00004H	0000CH
Usage	Debug each instruction	Halt at key points
Frequency	After every instruction	As and when INT 03 is invoked
Control	Automatic	Programmer-defined

Conclusion

- Interrupts Types 0–4 serve essential system functions and debugging support.
- Assembler programming provides precise control over hardware operations.
- Single Step and Breakpoint interrupts are powerful tools for software **debugging**, **testing**, and **system diagnostics** on the 8086.

Let me know if you want visual diagrams or flowcharts added to this!

Compare RISC and CISC Architectures with Examples

Feature	RISC (Reduced Instruction Set Computer)	CISC (Complex Instruction Set Computer)
Instruction Set	Few, simple instructions	Many, complex instructions
Execution Speed	Faster due to single-cycle instructions	Slower due to multi-cycle instructions
Code Size	Larger (more instructions for same task)	Smaller (fewer instructions)
Hardware Usage	Requires more RAM, simpler hardware	More complex hardware, optimized RAM use
Pipelining	Easier due to uniform instruction length	Difficult due to varied instruction sizes
Examples	ARM, MIPS, PowerPC	Intel x86, 8051, VAX

1. Define Embedded System. Explain its Characteristics with Examples

Definition of Embedded System:

An **embedded system** is a **special-purpose computing system** designed to perform **one or a few dedicated functions**, often embedded as part of a larger device. It includes a **microprocessor or microcontroller**, **application-specific software**, and **hardware components**.

"An embedded system uses a microprocessor (or microcontroller) to do one task. There is only one application software that is typically burned into ROM."

Characteristics of Embedded Systems:

Characteristic	Description	Example
Single-Functioned	Performs **one specific function repeatedly**.	Fire alarm only detects smoke.
Tightly Constrained	Limited **resources**, **power**, and **performance** optimized.	Heartbeat monitor in a smartwatch.
Reactive & Real-Time	Must respond quickly to **external inputs** with **real-time accuracy**.	Airbag deployment in cars.
Memory	Software is usually stored in **ROM** or flash memory.	Washing machine's firmware in ROM.
Connected	Interfaces with other systems through **I/O devices**, sensors, actuators, etc.	Industrial automation sensors.
HW-SW Co-Design	Combines **software flexibility** with **hardware efficiency and security**.	GPS module with map software.

2. Compare RISC and CISC Architectures with Examples

RISC (Reduced Instruction Set Computer)

Feature	Description
Instruction Set	Small and highly optimized. Few instructions used.

Feature	Description
Execution Speed	Faster — executes one instruction per cycle.
Instruction Length	Fixed-length instructions (usually 32 bits).
Addressing Modes	Fewer addressing modes.
Hardware Complexity	Simpler hardware, more control to software.
Example	ARM (used in mobile phones), **PIC 18F87X**

CISC (Complex Instruction Set Computer)

Feature	Description
Instruction Set	Large number of complex instructions.
Execution Speed	Slower — some instructions take multiple cycles.
Instruction Length	Variable-length instructions.
Addressing Modes	Many addressing modes.
Hardware Complexity	More complex hardware to decode instructions.
Example	Intel 8051, x86 architecture

From Document:

"Intel 8051 is an example of CISC machine, whereas PIC 18F87X is an example of RISC machine."

3. Differentiate Harvard and Von-Neumann Architectures

Harvard Architecture

Feature	Description
Memory Separation	Separate memory for instructions and data.
Bus System	Separate buses for data and instructions.
Execution Speed	Faster — can **fetch instructions and data simultaneously**.
Flexibility	Less flexible; memory units are fixed.
Example	**8051 microcontroller**, AVR microcontrollers.

Von-Neumann Architecture

Feature	Description
Memory Structure	Single memory for data and instructions.
Bus System	One common bus used for fetching data and instructions (bottleneck).
Execution Speed	Slower — must **alternate between data and instruction fetches**.
Flexibility	More flexible — same memory space reused.
Example	Traditional **x86** desktop processors, 8086.

Visual Hint (if diagrams are allowed in your answer):

- Harvard: Two separate memory units and buses.
- Von Neumann: One combined memory with a shared bus.

Summary Table: Harvard vs Von-Neumann

Feature	Harvard Architecture	Von-Neumann Architecture
Memory Structure	Separate instruction/data	Unified memory
Bus System	Dual buses	Single bus
Speed	Faster (parallel access)	Slower (sequential access)
Used In	Embedded systems, DSP	General-purpose processors
Example	AVR, PIC, 8051	Intel x86, ARM Cortex-A

4. Write an ALP (Assembly Language Program) to Count Number of 1s and 0s in a Byte

Objective:

Count the number of **set bits (1s)** and **clear bits (0s)** in an 8-bit number using **8051 Assembly Language**.

ALGORITHM:

1. Load the byte into the accumulator.
2. Use a loop to **rotate the accumulator right (RR A)** 8 times.
3. After each rotation:
   - Check the **carry flag** (CY).
   - If CY = 1 → increment ONE_COUNT.
   - Else → increment ZERO_COUNT.
4. Loop until all 8 bits are checked.

PROGRAM:

```
ORG 0000H

MOV A, #0A5H       ; Load byte A5H = 10100101
MOV R1, #08H       ; Loop counter = 8 bits
MOV R2, #00H       ; R2 = count of 1s
MOV R3, #00H       ; R3 = count of 0s

NEXT_BIT:
   RRC A          ; Rotate right through carry
   JC INCREMENT_1  ; If carry set, it's a 1
   INC R3         ; Increment zero count
   SJMP CONTINUE

INCREMENT_1:
   INC R2         ; Increment one count

CONTINUE:
   DJNZ R1, NEXT_BIT

; Now R2 has count of 1s and R3 has count of 0s
END
```

Explanation:

- RR or RRC: Rotate right with or without carry.
- JC: Jump if carry is set (i.e., bit was 1).
- DJNZ: Decrement and jump if not zero (loop control).

5. Explain the Memory Organization and Architecture of 8051

8051 Architecture Overview:

8051 is an **8-bit microcontroller** with a **Harvard architecture** (separate code and data memory).

Key Components:

- **8-bit ALU**
- **4 I/O Ports (P0 to P3)**
- **128 Bytes of RAM**
- **4 KB of ROM (in 8051)**
- **Timers: 2**
- **Serial port**
- **Interrupts: 5 (2 external + 3 internal)**

Memory Organization:

Memory Type	Size	Address Range	Description
Internal RAM	128 B	00H to 7FH	Includes register banks, bit-addressable RAM
ROM (Code)	4 KB	0000H to 0FFFH	Stores program code
Bit-addressable	16 B	20H to 2FH	128 bits, accessed via SETB, CLR
SFRs	128 B	80H to FFH	Special Function Registers (acc, TMOD, IE etc.)

Register Banks:

- 4 register banks (R0–R7)
- Default: Bank 0 (00H to 07H)
- Bank switching via **PSW register** (RS1 & RS0)

Stack Memory:

- Starts from 08H by default
- Used for temporary data during subroutine calls

I/O Ports:

- **Port 0**: Multiplexed Address/Data (external memory)
- **Port 1**: General I/O
- **Port 2**: Higher address byte (if external memory used)
- **Port 3**: I/O with alternate functions (INT0, INT1, TxD, RxD)

6. Compare 4-bit, 8-bit, 16-bit, and 32-bit Microcontrollers

Microcontrollers are classified based on **word size** — the number of bits processed at once.

Feature	4-bit	8-bit	16-bit	32-bit
Data Bus Width	4 bits	8 bits	16 bits	32 bits
Processing Speed	Very low	Moderate	Fast	Very fast
Memory Access	Limited	Up to 64KB	64KB to 1MB	>4GB
Instruction Set	Very simple	Moderate	Complex	Highly optimized
Applications	Simple watches, toys	Consumer electronics, 8051	Automotive, industrial control	IoT, mobile, AI, ARM Cortex-M
Example	TMS1000	Intel 8051, PIC 16F877A	MSP430, 80196	ARM Cortex-M, STM32, ESP32
Cost	Very low	Low	Moderate	Higher

Use Cases:

- **4-bit**: Calculators, old digital clocks
- **8-bit**: Keyboards, basic sensors, small robots
- **16-bit**: Industrial automation, motor control
- **32-bit**: Embedded Linux systems, smart devices, IoT controllers

7. Write an ALP to Perform Data Transfer Between Two Memory Locations

Objective:

Transfer a byte from **Source Memory Location** (e.g., 40H) to **Destination Memory Location** (e.g., 50H).

Assembly Language Program (8051):

```
ORG 0000H

; Load the source address
MOV R0, #40H      ; R0 points to source address

; Load the destination address
MOV R1, #50H      ; R1 points to destination address

; Move data from source to accumulator
MOV A, @R0        ; A = [40H]

; Store the data to destination
MOV @R1, A        ; [50H] = A

END
```

Explanation:

- MOV R0, #40H sets pointer to source.
- MOV A, @R0 fetches data from indirect address.
- MOV @R1, A stores it at destination.
- Uses **indirect addressing** with registers R0 and R1.

8. Write an ALP to Exchange N Bytes of Data Between Two Locations

Objective:

Exchange N bytes of data between memory locations starting from **LOC1** and **LOC2**.

8051 ALP to Exchange N Bytes:

```
ORG 0000H

MOV R2, #05H      ; N = 5 bytes to exchange
MOV R0, #30H      ; R0 points to LOC1
MOV R1, #40H      ; R1 points to LOC2

EXCHANGE_LOOP:
   MOV A, @R0     ; Load byte from LOC1
   MOV B, @R1     ; Load byte from LOC2

   MOV @R0, B     ; Store LOC2 data into LOC1
   MOV @R1, A     ; Store LOC1 data into LOC2

   INC R0         ; Next byte in LOC1
   INC R1         ; Next byte in LOC2

   DJNZ R2, EXCHANGE_LOOP

END
```

Explanation:

- Exchanges data between blocks starting at 30H and 40H.
- Loop runs for 5 bytes (R2).
- Uses **register indirect addressing mode** for memory operations.

9. Explain Different Addressing Modes in 8051 with Examples

Addressing modes define how an **instruction refers to operands**. 8051 supports the following:

1. Immediate Addressing Mode

- Operand is a constant, prefixed by #.
- **Syntax**: MOV A, #0AH
- **Explanation**: Load immediate value into accumulator.

2. Register Addressing Mode

- Operand is a register (R0–R7).
- **Syntax**: MOV A, R3
- **Explanation**: Move data from register R3 to accumulator.

3. Direct Addressing Mode

- Specifies a direct RAM or SFR address.

- **Syntax**: MOV A, 30H
- **Explanation**: Move contents of internal RAM location 30H into accumulator.

4. Indirect Addressing Mode

- Uses R0 or R1 to point to memory location.
- **Syntax**: MOV A, @R0
- **Explanation**: Load accumulator with the value stored at the address held in R0.

5. Indexed Addressing Mode

- Used to access code memory (ROM) for **lookup tables**.
- Combines **DPTR** or **PC** with accumulator.
- **Syntax**: MOVC A, @A+DPTR
- **Explanation**: A = ROM[(DPTR + A)] — useful in table reads.

Summary Table:

Addressing Mode	Syntax Example	Description
Immediate	MOV A, #55H	Constant loaded into register
Register	MOV A, R1	Data from general-purpose register
Direct	MOV A, 30H	Access internal RAM directly
Indirect	MOV A, @R0	Address in R0 used to access memory
Indexed	MOVC A, @A+DPTR	Fetch from code memory using base + offset

From the Document:

"8051 addressing modes are classified as Immediate, Register, Direct, Indirect, and Indexed."

10. Write an ALP to Add Two 16-bit Numbers in 8051

Objective:

Add two 16-bit numbers stored in memory (e.g., 3000H and 3002H), and store the result at another location (e.g., 3004H).

8051 Assembly Language Program:

```
ORG 0000H

; Load lower byte of first number
MOV DPTR, #3000H
MOVX A, @DPTR
MOV R0, A        ; R0 = low byte of first number

; Load higher byte of first number
INC DPTR
MOVX A, @DPTR
MOV R1, A        ; R1 = high byte of first number

; Load lower byte of second number
MOV DPTR, #3002H
```

```
MOVX A, @DPTR
MOV R2, A        ; R2 = low byte of second number

; Load higher byte of second number
INC DPTR
MOVX A, @DPTR
MOV R3, A        ; R3 = high byte of second number

; Add lower bytes
MOV A, R0
ADD A, R2
MOV R4, A        ; R4 = result lower byte

; Add higher bytes with carry
MOV A, R1
ADDC A, R3
MOV R5, A        ; R5 = result higher byte

; Store result at 3004H
MOV DPTR, #3004H
MOV A, R4
MOVX @DPTR, A

INC DPTR
MOV A, R5
MOVX @DPTR, A

END
```

Explanation:

- MOVX: Access external memory.
- ADD/ADDC: Adds with and without carry.
- Result stored in memory from address 3004H.

11. Explain Features and Architecture of the 8051 Microcontroller

Features of 8051:

Feature	Description
8-bit CPU	ALU and registers handle 8-bit data.
On-chip ROM	4 KB program memory.
On-chip RAM	128 bytes internal RAM.
I/O Ports	4 parallel ports (P0 to P3), each 8-bit wide.
Timers	Two 16-bit timers (Timer 0 and Timer 1).
Serial Port	Full duplex UART communication.
Interrupts	5 interrupt sources (2 external, 3 internal).
Clock Frequency	Typically 12 MHz.

8051 Architecture Diagram (Describe if not drawing):

Key Components:

- **CPU**: 8-bit, fetches, decodes and executes instructions.
- **Registers**: General purpose (R0-R7), Special Function Registers (SFRs).
- **Memory**:
  - 4 KB ROM for program.
  - 128 B RAM for data and stack.
- **I/O Ports**: P0–P3 (used for interfacing external devices).
- **Timers**: T0 and T1 – used for delay generation, counting.
- **Serial Port**: Enables RS232-based communication.
- **Interrupt Control**: Handles interrupts using vector table and priority.

Segment from Document:

"The 8051 microcontroller has 128 bytes of internal RAM, 4K of ROM, 2 timers, 32 I/O lines, serial port, and interrupts."

1. Explain the TMOD Register and Its Role in Timer Operation

TMOD Register (Timer Mode Register):

- A **Special Function Register (SFR)** used to configure the operation mode of **Timer 0** and **Timer 1**.
- Address: **89H** (TMOD).

TMOD Format:

Bit No	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Name	GATE1	C/T1	M1_1	M0_1	GATE0	C/T0	M1_0	M0_0

- **GATE**: Controls external INTx pin as a start/stop for timer.
- **C/T**: Selects **Timer** (0) or **Counter** (1) mode.
- **M1 & M0**: Mode selection bits (for each timer).

Timer Modes:

M1	**M0**	**Mode**	**Description**
0	0	Mode 0	13-bit Timer
0	1	Mode 1	16-bit Timer
1	0	Mode 2	8-bit auto-reload Timer
1	1	Mode 3	Split timer mode (Timer 0 only)

Example:

To set Timer 1 in **Mode 1 (16-bit timer)**:

```
MOV TMOD, #10H  ; GATE1=0, C/T1=0, M1_1=0, M0_1=1
```

Role of TMOD in Timer Operation:

- **Selects mode** of operation (delay, pulse generation, counting).
- **Chooses source**: internal clock (Timer) or external signal (Counter).
- **Enables software/hardware trigger** via GATE.

2. Describe Mode 1 and Mode 2 Programming in 8051

The **8051 microcontroller** supports four timer modes. Among them, **Mode 1** and **Mode 2** are widely used.

Mode 1 – 16-bit Timer Mode

- **Description**: Timer functions as a **16-bit counter**, with THx (high byte) and TLx (low byte).
- **Range**: 0000H to FFFFH (65536 counts).
- **Typical Use**: Longer delays or event counting.
- **Mode Selection Bits**: M1 = 0, M0 = 1

Steps to Program Mode 1:

- Load TMOD with 00000001B for Timer 0 (Mode 1).
- Load initial values into TH0 and TL0.
- Set TR0 (Timer Run bit).
- Monitor **TF0** (Timer Flag 0).
- Stop timer when **TF0 = 1**, then clear it.

Mode 2 – 8-bit Auto-Reload Mode

- **Description**: Timer operates as an **8-bit timer** (only TLx used).
- **Auto-reload**: TLx reloads from THx after overflow.
- **Range**: 00H to FFH (256 counts).
- **Mode Selection Bits**: M1 = 1, M0 = 0
- **Use Case**: Continuous time delay (e.g., baud rate generator).

Steps to Program Mode 2:

1. Set TMOD to 00000010B (for Timer 0).
2. Load the **reload value** into TH0.
3. Start timer by setting **TR0**.
4. When TL0 overflows (FF → 00), TF0 is set.
5. TL0 is **automatically reloaded** from TH0.

Comparison Table:

Feature	Mode 1	Mode 2
Timer Width	16-bit (TH + TL)	8-bit (TL only, reloads from TH)
Reload Mechanism	Manual (TH, TL reloaded)	Automatic from TH
Use Case	Long, precise delays	Repetitive short delays

3. Write a Program to Generate a Time Delay Using Timer 0

Objective: Create a **delay using Timer 0 in Mode 1** (16-bit timer) assuming 12 MHz crystal.

Delay Calculation:

- 8051 takes **1 machine cycle = 12 clock cycles**.
- For 12 MHz crystal:
  1 cycle = 1 μs → 65536 counts = 65.536 ms

Program to Generate 1ms Delay Using Timer 0:

```
ORG 0000H

; Set Timer 0 to Mode 1
MOV TMOD, #01H      ; Timer 0, Mode 1

; Load Timer with count (65536 - 1000 = 64536 = FC18H)
MOV TH0, #0FCH      ; High byte
MOV TL0, #018H      ; Low byte

SETB TR0            ; Start Timer 0

WAIT:
JNB TF0, WAIT       ; Wait until TF0 is set (overflow)

CLR TR0             ; Stop Timer
CLR TF0             ; Clear overflow flag

; Delay Complete - continue program

END
```

Explanation:

- TMOD = 01H: Timer 0, Mode 1.
- TH0:TL0 = FC18H → 1000 counts (1 ms).
- Loop waits for **TF0 = 1** (overflow).
- TF0 cleared and timer stopped after delay.

4. Describe the Importance of Timer Flag and How It's Used

What is a Timer Flag (TF0/TF1)?

- Flags used to **indicate overflow** in Timer 0 or Timer 1.
- **TF0**: Timer Flag 0 (for Timer 0).
- **TF1**: Timer Flag 1 (for Timer 1).
- Part of the **TCON (Timer Control Register)** at address 88H.

Role of Timer Flag:

Function	Description
Set on overflow	When timer counts from max value (FFFFH/FFH) → 0, TF is **set to 1**.
Signals end of delay	Used to signal to software that the delay cycle is complete.
Manually cleared	Must be cleared by software using CLR TFx after servicing the event.

How It's Used in Programming:

```
JNB TF0, HERE     ; Jump if TF0 not set → wait
CLR TF0           ; Clear TF0 after it is set (overflow occurred)
```

- TF0 acts like a **polling flag**.

- Can also be used to **trigger interrupts** if ET0 and EA are enabled.

Key Point from Document: *"When timer overflows, TF is set. This can be used in polling or interrupt mode to identify when timer has completed."*

5. Explain Doubling the Baud Rate in 8051 Using SMOD Bit

What is Baud Rate?

Baud rate refers to the **number of bits transmitted per second** in serial communication.

In 8051, baud rate is typically derived from **Timer 1**, especially in **Mode 2 (auto-reload mode)**.

Baud Rate Formula in 8051:

Baud Rate=2SMOD32×(Oscillator Frequency12×(256−TH1))
SMOD (Serial Mode Bit): Bit 7 of the **PCON register (Power Control Register)**.

- **TH1**: Timer 1 reload value (used in Mode 2).
- **Oscillator Frequency**: Usually 11.0592 MHz or 12 MHz.

Effect of SMOD Bit:

SMOD	Baud Rate Effect
0	Normal baud rate
1	**Doubles** the baud rate

To double the baud rate:

```
PCON |= 0x80;  // Set SMOD bit to 1
```

Example: To achieve **9600 bps** using 11.0592 MHz oscillator:

- With **SMOD = 0**: TH1 = 253 (FDH)
- With **SMOD = 1**: TH1 = 254 (FEH) → Doubled rate

Key Point: Setting SMOD = 1 in PCON register allows **higher baud rates** without changing oscillator frequency.

6. Write a Program to Calculate Delay Using Timers

Objective: Create a time delay using **Timer 1 in Mode 1** (16-bit timer).

Assume 11.0592 MHz crystal, and we want ~50 ms delay.

Timer Delay Basics:

1 machine cycle = 12 clock cycles

Machine cycle time=111.0592×106×12≈1.085μs

To get 50 ms:

$50 \times 10^{-3} / 1.085 \times 10^{-6} \approx 46080$ counts
$65536 - 46080 = 19456 = 4C00H$

Assembly Code:

```
ORG 0000H

MOV TMOD, #10H     ; Timer1, Mode1
MOV TH1, #04H      ; High byte of 4C00H
MOV TL1, #C0H      ; Low byte

SETB TR1           ; Start Timer1

WAIT:
JNB TF1, WAIT      ; Wait until overflow

CLR TR1            ; Stop timer
CLR TF1            ; Clear overflow flag

; Delay complete

END
```

Explanation:

- TMOD = 10H: Sets Timer 1 in Mode 1.
- Timer runs for 50 ms, then TF1 sets.
- Timer stopped and flag cleared after delay.

7. Explain Serial Port Programming with 8051 in C

Serial Communication in 8051

8051 supports **UART-based serial communication** using **SBUF**, **SCON**, **TI**, and **RI** registers.

Steps in Serial Port Programming:

- **Configure Timer 1** (for baud rate):
  1. Use Mode 2 (auto-reload).
  2. Set TH1 for desired baud rate.
- **Set SCON**:
  1. SCON = 0x50 → Mode 1, 8-bit data, 1 stop bit, REN enabled.
- **Enable SMOD (optional)**:
  1. PCON |= 0x80 to double baud rate.
- **Use SBUF**:
  1. Write to SBUF to **transmit**.
  2. Read from SBUF to **receive**.
- **Monitor TI and RI flags**:
  1. TI = 1 → transmission complete.
  2. RI = 1 → receive complete.

C Code Example: Transmit a Character 'A'

```
#include <reg51.h>

void serial_init() {
   TMOD = 0x20;     // Timer 1, Mode 2
   TH1 = 0xFD;      // Baud rate 9600
   SCON = 0x50;     // 8-bit UART, REN enabled
   TR1 = 1;         // Start Timer 1
}

void transmit_char(char ch) {
   SBUF = ch;       // Load data to SBUF
   while (TI == 0); // Wait until transmission complete
   TI = 0;          // Clear transmit interrupt flag
}

void main() {
   serial_init();
   transmit_char('A');  // Send ASCII 'A'
   while (1);
}
```

Explanation:

- serial_init() sets up timer and serial port.
- transmit_char() sends a byte.
- TI flag ensures transmission is complete.

From Document: *"8051 uses SBUF for serial transfer. TI and RI are status flags for transmit/receive completion respectively."*

8. Compare Polling vs Interrupt in Timers

Timers in the 8051 microcontroller can be monitored using either **polling** or **interrupt-driven** techniques.

Polling Method

Feature	Description
How it works	Continuously checks the **Timer Flag (TF)** in a loop.
CPU Usage	CPU is **busy waiting** → cannot do other tasks during delay.
Code Simplicity	Easier to implement.
Response Time	Deterministic, but **not efficient**.
Suitable for	Simple systems or when multitasking is not required.

Example:

```
JNB TF0, $   ; Wait here until TF0 is set
CLR TF0      ; Clear flag after delay
```

Interrupt Method

Feature	Description
How it works	Timer sets **TF**, which triggers **interrupt service routine (ISR)**.
CPU Usage	CPU continues other work; **interrupts only when needed**.
Code Complexity	Slightly more complex due to ISR and flags.
Response Time	Fast and efficient.
Suitable for	**Multitasking or real-time** applications.

Example (in C):

```
void timer0_ISR(void) interrupt 1 {
   // This runs automatically when TF0 = 1
   TF0 = 0;
}
```

Summary Table:

Aspect	Polling	Interrupt
CPU Efficiency	Low (busy waiting)	High (CPU free until needed)
Code Simplicity	Simple	Requires ISR setup
Real-Time Use	Not ideal	Highly suitable
Flexibility	Less flexible	More flexible and multitasking capable

9. Describe Steps to Program Timer 1 in 8051

Timer 1 can be programmed in several modes, most commonly **Mode 1 (16-bit)** or **Mode 2 (8-bit auto-reload)**.

Steps to Program Timer 1 (e.g., in Mode 1):

Step-by-Step Procedure:

- **Select Mode via TMOD Register**
    1. Timer 1 is controlled by upper nibble of TMOD.
    2. For Mode 1 (16-bit):
    3. MOV TMOD, #00010000B ; (10H)
- **Load Initial Count into TH1 and TL1**
    1. Based on required delay.
    2. MOV TH1, #HIGH_BYTE
    3. MOV TL1, #LOW_BYTE
- **Start the Timer by Setting TR1**
- SETB TR1
- **Wait for Overflow (Polling) or Enable Interrupt**
    1. Polling:
    2. JNB TF1, $
    3. Or: Enable **ET1**, **EA** for interrupt.
- **Stop Timer (optional)**
- CLR TR1
- **Clear Timer Flag TF1**
- CLR TF1

Example (Polling Based) for 50 ms:

```
MOV TMOD, #10H
MOV TH1, #04H
MOV TL1, #C0H
SETB TR1
JNB TF1, $
CLR TR1
CLR TF1
```

10. Write a Program to Generate a Square Wave Using Timer

Objective:

Toggle a pin (e.g., **P1.0**) repeatedly using **Timer 1** to generate a **square wave**.

Concept:

- Use **Timer 1 Mode 1** to create half of the wave period (high or low).
- Toggle **P1.0** each time the timer overflows.
- For a 1 ms HIGH and 1 ms LOW → 2 ms period → 500 Hz square wave.

Assembly Program:

```
ORG 0000H

START:
   MOV TMOD, #10H      ; Timer 1, Mode 1
   MOV P1, #00H        ; Clear port
LOOP:
   MOV TH1, #0FCH      ; Load high byte (for ~1 ms)
   MOV TL1, #018H      ; Load low byte
   SETB TR1            ; Start Timer 1

WAIT:
   JNB TF1, WAIT       ; Wait for overflow
   CLR TF1             ; Clear overflow flag
   CLR TR1             ; Stop timer

   CPL P1.0            ; Toggle pin P1.0

   SJMP LOOP           ; Repeat forever
END
```

Explanation:

- TH1:TL1 = FC18H ≈ 1000 cycles ≈ 1 ms delay
- CPL P1.0: Complements (toggles) pin state
- Repeats forever to generate continuous square wave

Can be done in C:

```
#include <reg51.h>

void delay() {
   TMOD = 0x10;      // Timer 1, Mode 1
   TH1 = 0xFC;
   TL1 = 0x18;
   TR1 = 1;
   while (TF1 == 0);
   TR1 = 0;
   TF1 = 0;
}

void main() {
   while (1) {
      P1 ^= 0x01;   // Toggle P1.0
      delay();      // Delay ~1 ms
   }
}
```

1. Explain the Interrupt Structure of 8051 with IVT (Interrupt Vector Table)

Definition of Interrupt

An **interrupt** is an event that temporarily halts the main program execution so the microcontroller can execute a separate piece of code known as an **Interrupt Service Routine (ISR)**.

Interrupt Structure of 8051:

8051 supports **5 interrupt sources**, prioritized and vectored through fixed memory locations (IVT – Interrupt Vector Table).

Interrupt Source	Vector Address	Priority	Triggered By
INT0 (External 0)	0003H	1	External signal (P3.2)
Timer 0 Overflow	000BH	2	Timer 0 overflow
INT1 (External 1)	0013H	3	External signal (P3.3)
Timer 1 Overflow	001BH	4	Timer 1 overflow
Serial (RI/TI)	0023H	5	Serial Tx or Rx complete

Interrupt Vector Table (IVT):

IVT contains **addresses** of each ISR. When an interrupt occurs, the 8051 jumps to the **fixed vector address**.

Interrupt	Vector Address	ISR Location
Reset	0000H	Startup code
INT0	0003H	ISR for INT0
T0	000BH	ISR for Timer0
INT1	0013H	ISR for INT1

Interrupt	Vector Address	ISR Location
T1	001BH	ISR for Timer1
Serial	0023H	ISR for Serial port

Execution Flow:

- Interrupt occurs → microcontroller completes current instruction.
- Pushes **PC** on stack.
- Loads new **PC from vector address**.
- Executes ISR.
- Returns using RETI instruction.

2. Describe Various Types of Interrupts Supported by 8051

8051 supports **5 interrupts**, categorized as **external**, **internal**, **hardware**, and **software** interrupts.

1. External Interrupts

- **INT0**: Triggered via pin **P3.2**.
- **INT1**: Triggered via pin **P3.3**.
- Configurable as **level- or edge-triggered** using **TCON register (IT0, IT1 bits)**.

2. Timer Interrupts

- **Timer 0 and Timer 1**: Triggered when timers **overflow**.
- TF0 and TF1 bits in **TCON** are set.
- Used for delay, event counting, pulse generation.

3. Serial Interrupt

- Triggered when either:
  - **RI (Receive Interrupt)** is set.
  - **TI (Transmit Interrupt)** is set.
- Both are in **SCON register**.
- Used in **UART communication**.

Interrupt Types Summary Table:

Interrupt Type	Source	Vector	Trigger Condition
External	INT0, INT1	0003H, 0013H	Edge or level (via TCON)
Timer	Timer0, Timer1	000BH, 001BH	Overflow (TF0/TF1)
Serial	RI/TI (UART)	0023H	Byte received/transmitted

From Document: *"8051 microcontroller supports 5 interrupts: 2 external, 2 timer, and 1 serial port interrupt."*

3. Explain Enabling and Disabling of Interrupts in 8051

Interrupts in 8051 are controlled through the **Interrupt Enable (IE)** and **Interrupt Priority (IP)** registers.

1. IE (Interrupt Enable Register)

Bit	Symbol	Description
7	EA	**Global enable/disable** (1 = enable)
6	—	Reserved
5	ES	Serial interrupt enable
4	ET1	Timer 1 interrupt enable
3	EX1	External interrupt 1 enable
2	ET0	Timer 0 interrupt enable
1	EX0	External interrupt 0 enable
0	—	Reserved

Example: Enable Timer 0 and External 0

```
MOV IE, #10000101B  ; EA=1, EX0=1, ET0=1
```

2. Enabling Interrupts

- To enable a specific interrupt:
  - Set **EA = 1** (global enable).
  - Set the corresponding bit (e.g., ET0 = 1).

Example (C Code):

```
IE = 0x82; // EA=1, EX0=1
```

3. Disabling Interrupts

- Clear the relevant bit or **clear EA**.
- Setting EA = 0 disables **all interrupts**, regardless of individual bits.

Example:

```
CLR EA  ; Disable all interrupts
```

4. Interrupt Priority Control – IP Register

Bit	Symbol	Description
7	—	Reserved
6	—	Reserved
5	PS	Serial interrupt priority
4	PT1	Timer 1 priority
3	PX1	External 1 priority
2	PT0	Timer 0 priority
1	PX0	External 0 priority
0	—	Reserved

- **1 = High priority**, **0 = Low priority**
- If two interrupts occur simultaneously, **high-priority is serviced first**.

Summary Table: Enabling vs Disabling

Action	Instruction/Bit
Enable all	SETB EA or `IE
Disable all	CLR EA or IE &= 0x7F
Enable INT0	SETB EX0 (bit 0 of IE)
Disable T1	CLR ET1 (bit 4 of IE)

4. Describe External and Timer Interrupts in Detail

Overview of 8051 Interrupt System:

8051 has **5 interrupt sources**. Among these, **2 are external (INT0, INT1)** and **2 are internal timer-based (T0, T1)**.

External Interrupts (INT0 and INT1)

Name	Vector Address	Pin Used	Configurable	Control Bit	Use Case
INT0	0003H	P3.2	Edge/Level	EX0 (IE.0)	External button, pulse input
INT1	0013H	P3.3	Edge/Level	EX1 (IE.3)	Emergency stop, sensor input

Configuration:

- Use **TCON Register**:
  - IT0 (bit 0): INT0 type → 1 = edge, 0 = level
  - IT1 (bit 2): INT1 type → same as above
- Use **IE Register**:
  - EX0 = 1 → enable INT0
  - EX1 = 1 → enable INT1
  - EA = 1 → global enable

Example (Enable INT0, edge triggered):

```
IE = 0x81;   // EA=1, EX0=1
TCON |= 0x01; // IT0 = 1 → edge triggered
```

Timer Interrupts (Timer 0 and Timer 1)

Name	Vector Address	Flag	Enable Bit (IE)	Priority Bit (IP)	Use Case
Timer 0	000BH	TF0	ET0 (IE.1)	PT0 (IP.1)	Delay, counters
Timer 1	001BH	TF1	ET1 (IE.4)	PT1 (IP.4)	Serial, delay loops

Timer Interrupt Flow:

- Load TMOD for desired mode (usually Mode 1 or 2).
- Load THx/TLx for desired delay.
- Start timer by setting TRx.
- When timer overflows:
  1. TFx is set.
  2. If ETx and EA are enabled, ISR is invoked.

Sample ISR for Timer 0 (Assembly):

```
ORG 000BH
TIMER0_ISR:
  CLR TF0
  ; Your code here
  RETI
```

5. Write a C Program to Handle Serial Interrupts

Objective:

Create a C program for 8051 to **send and receive characters** using **serial interrupt**.

Program (Keil C) – Serial Interrupt Handler:

```
#include <reg51.h>

void serial_ISR(void) interrupt 4 {
    if (RI) {
        RI = 0;                 // Clear receive flag
        P1 = SBUF;              // Display received char on P1
    }
    if (TI) {
        TI = 0;                 // Clear transmit flag
    }
}

void main() {
    SCON = 0x50;      // Mode 1, 8-bit UART, REN enabled
    TMOD = 0x20;      // Timer1 Mode 2
    TH1 = 0xFD;       // Baud rate 9600
    TL1 = 0xFD;
    TR1 = 1;          // Start Timer1
    IE = 0x90;        // EA=1, ES=1 (Enable serial interrupt)

    while (1) {
        SBUF = 'A';   // Send 'A'
        while (!TI);  // Wait for transmission complete
        TI = 0;
    }
}
```

Explanation:

- interrupt 4 → Serial ISR
- RI = Receive interrupt flag; TI = Transmit flag
- Serial is configured for 9600 baud
- ISR handles both Rx and Tx

6. Write a Program to Generate Sine, Ramp, and Triangular Waveforms

Hardware Requirement:

- **DAC (e.g., DAC0808)** connected to 8051 via an 8-bit port (e.g., P1)
- **Waveform generation** by sending pre-calculated digital values to DAC

C Code for 8051 + DAC0808:

1. Sine Wave Generation:

```
#include <reg51.h>

unsigned char sine[] = {
  128, 152, 176, 198, 218, 234, 245, 251, 255, 251, 245,
  234, 218, 198, 176, 152, 128, 104, 80, 58, 38, 22, 11,
  5, 1, 5, 11, 22, 38, 58, 80, 104
}; // 32 values of sine wave

void delay() {
  int i;
  for(i=0; i<500; i++);
}

void main() {
  int i;
  while(1) {
    for(i=0; i<32; i++) {
      P1 = sine[i];  // Output to DAC
      delay();
    }
  }
}
```

2. Ramp Waveform:

```
void main() {
  unsigned char i;
  while(1) {
    for(i=0; i<255; i++) {
      P1 = i;
      delay();
    }
  }
}
```

3. Triangular Waveform:

```
void main() {
  unsigned char i;
  while(1) {
    for(i=0; i<255; i++) {
      P1 = i;
      delay();
    }
    for(i=255; i>0; i--) {
      P1 = i;
      delay();
```

```
    }
  }
}
```

Explanation:

- **P1** is connected to DAC input lines.
- Delay functions control waveform frequency.
- Data sent to DAC gets converted to analog output.

7. Explain the Concept of ISR and Steps Involved in Execution

What is ISR (Interrupt Service Routine)?

An **Interrupt Service Routine (ISR)** is a special block of code that executes **automatically** in response to an **interrupt**.

- It is **invoked by hardware** or software signals.
- Each ISR is **associated with a specific interrupt vector address**.
- Ends with RETI (Return from Interrupt).

Steps in ISR Execution in 8051:

1. **Interrupt Occurs**:
   - Could be external (INT0/INT1), timer overflow, or serial event.
2. **Current Instruction Completes**:
   - 8051 finishes executing the current instruction.
3. **Program Counter (PC) is Saved**:
   - **Pushes PC onto the stack** (2 bytes).
   - Saves return address.
4. **Control Jumps to ISR Vector Address**:
   - Example: For INT0, jumps to address 0003H.
5. **ISR Code Executes**:
   - Handles the event, like reading data or resetting a flag.
6. **RETI Instruction Executes**:
   - **Pops PC from the stack**, resumes execution from where it was interrupted.

Features of ISR:

- Cannot return values.
- Must be **short and fast**.
- Must end with RETI (Return from Interrupt).
- **Nested interrupts** can be enabled using the **IP (Interrupt Priority) register**.

Example: Timer 0 ISR (Assembly):

```
ORG 000BH        ; Timer 0 ISR vector
TIMER0_ISR:
   CLR TF0       ; Clear timer flag
   INC P1        ; Example action
   RETI          ; Return from interrupt
```

8. Explain LCD Interfacing with 8051 and Pin Configuration

Overview: LCD (e.g., 16×2) is used to display characters/numbers. It's interfaced with 8051 using **I/O ports**.

LCD Pin Configuration (16x2 LCD):

Pin	Name	Function
1	VSS	GND
2	VDD	+5V
3	VEE	Contrast control (via potentiometer)
4	RS	Register Select (0 = command, 1 = data)
5	RW	Read/Write (0 = Write, 1 = Read)
6	EN	Enable (latch data on high-to-low)
7–14	D0–D7	Data pins
15	LED+	Backlight +
16	LED-	Backlight -

Connections to 8051 (Example):

- Data lines **D0–D7** → Port 2 (P2.0 – P2.7)
- Control pins:
  - **RS** → P3.0
  - **RW** → P3.1
  - **EN** → P3.2

Interfacing Logic:

1. Set RS = 0 → Send **Command**
2. Set RS = 1 → Send **Data**
3. RW = 0 → **Write** mode
4. Pulse EN = 1 → 0 → Latch command/data

LCD Initialization Commands:

Command	Function
0x38	8-bit, 2-line display
0x0C	Display ON, Cursor OFF
0x06	Entry mode, auto-increment
0x01	Clear display

Example Code (C):

```
sbit RS = P3^0;
sbit RW = P3^1;
sbit EN = P3^2;

void lcd_cmd(unsigned char cmd) {
   P2 = cmd;
   RS = 0;
   RW = 0;
   EN = 1;
```

```
    delay();
    EN = 0;
}

void lcd_data(unsigned char data) {
    P2 = data;
    RS = 1;
    RW = 0;
    EN = 1;
    delay();
    EN = 0;
}

void main() {
    lcd_cmd(0x38); // Init LCD
    lcd_cmd(0x0C); // Display ON
    lcd_cmd(0x01); // Clear
    lcd_data('A'); // Display A
}
```

9. Explain Priority of Interrupts and the IP Register

What is Interrupt Priority?

When multiple interrupts occur **simultaneously**, the microcontroller services the one with the **highest priority** first.

By default:

- INT0 has highest priority
- Serial interrupt has the lowest

But using the **IP register**, priorities can be changed.

IP (Interrupt Priority) Register

Bit	Symbol	Description
7	–	Reserved
6	–	Reserved
5	PS	Serial Port priority
4	PT1	Timer 1 priority
3	PX1	External 1 priority
2	PT0	Timer 0 priority
1	PX0	External 0 priority
0	–	Reserved

- **1 = High priority**, **0 = Low priority**
- If two interrupts occur at once:
    - High priority interrupt is serviced first.
    - If both are same level, default vector order is followed.

Example: Make Timer 1 High Priority:

MOV IP, #00010000B ; PT1 = 1 → Timer 1 is high priority

Nested Interrupts:

- High-priority interrupts can **interrupt a low-priority ISR**.
- A low-priority ISR **cannot interrupt a high-priority ISR**.

Summary Table:

Interrupt Source	IP Bit	Default Priority	Can be Changed?
INT0	PX0	Highest	Yes
Timer 0	PT0	High	Yes
INT1	PX1	Medium	Yes
Timer 1	PT1	Medium	Yes
Serial	PS	Lowest	Yes

10. Describe the Interfacing of a Stepper Motor with 8051

What is a Stepper Motor?

A **stepper motor** is a digital motor that moves in discrete steps. Each input pulse moves the motor shaft by a **fixed angle** (step angle).

- Common types: **Unipolar** and **Bipolar**
- Control via **transistor drivers** or **ULN2003A driver IC**

Connection Overview:

Component	Connected To
Motor coils	To outputs of ULN2003 (or transistor driver)
ULN2003 inputs	To 8051 port (e.g., P2.0–P2.3)
VCC & GND	Power supply for motor and logic

Why Use a Driver?

- 8051 cannot supply enough **current** to drive the motor.
- ULN2003 provides **high current** switching and protects the microcontroller.

Stepper Motor Sequence (Full Step):

Step	Coil Pattern (P2.3–P2.0)
1	1000
2	0100
3	0010
4	0001

8051 C Code for Clockwise Rotation:

```
#include <reg51.h>

void delay() {
  int i, j;
  for(i = 0; i < 100; i++)
    for(j = 0; j < 100; j++);
}

void main() {
  while(1) {
    P2 = 0x08; delay();  // 1000
    P2 = 0x04; delay();  // 0100
    P2 = 0x02; delay();  // 0010
    P2 = 0x01; delay();  // 0001
  }
}
```

Applications:

- Robotics
- CNC machines
- 3D printers
- Precision control systems

11. Differentiate Edge-Triggered and Level-Triggered Interrupts

Interrupts in 8051 can be either **edge-triggered** or **level-triggered**. This defines **how the interrupt is recognized** by the microcontroller.

Comparison Table:

Feature	Edge-Triggered	Level-Triggered
Trigger Type	Rising/Falling edge (transition)	High or Low logic level (constant)
8051 Support	Yes (INT0/INT1 configurable)	Yes (default if IT0/IT1 = 0)
Config Bit	IT0 / IT1 = 1 in TCON	IT0 / IT1 = 0 in TCON
Sensitivity	Detects **pulse change** only	Detects **constant high/low**
Interrupt Repetition	Needs a new edge	Interrupts as long as level is active
Example Use	Push button, external pulses	Alarm inputs, signal stays high

Example: Enable INT0 as Edge-Triggered:

```
TCON |= 0x01;  // IT0 = 1 → edge triggered
IE |= 0x81;    // EA = 1, EX0 = 1
```

12. Write an 8051 C Code to Generate Waveforms Using DAC

Requirements:

- **DAC0808** or **R-2R Ladder DAC**
- DAC connected to 8051 Port (e.g., P1)

- Delay function controls **frequency** of waveform

1. Sine Wave (Using lookup table):

```
#include <reg51.h>

unsigned char sine[32] = {
128,152,176,198,218,234,245,251,
255,251,245,234,218,198,176,152,
128,104, 80, 58, 38, 22, 11,  5,
  1,  5, 11, 22, 38, 58, 80,104
};

void delay() {
  int i,j;
  for(i=0;i<100;i++)
    for(j=0;j<50;j++);
}

void main() {
  int i;
  while(1) {
    for(i=0;i<32;i++) {
      P1 = sine[i];  // Send to DAC
      delay();
    }
  }
}
```

2. Ramp Waveform:

```
void main() {
  unsigned char i;
  while(1) {
    for(i=0; i<255; i++) {
      P1 = i;
      delay();
    }
  }
}
```

3. Triangular Waveform:

```
void main() {
  unsigned char i;
  while(1) {
    for(i=0; i<255; i++) {
      P1 = i;
      delay();
    }
    for(i=255; i>0; i--) {
      P1 = i;
      delay();
    }
```

```
  }
}
```

Explanation:

- **P1** → Connected to DAC input pins
- **delay()** → Adjusts waveform frequency
- DAC converts digital output from 8051 to analog voltage

Applications of DAC Waveform Generation:

- Signal generators
- Audio wave generation
- Function generator circuits